



PROJECT UPDATE-3 REPORT

GROUP	:	1	
COURSE	:	CSE299	
SECTION	:	3	
FACULTY	:	MSRB(Dr.M.Shifat-E-Rabbi)	
DEPARTMENT	:	ECE	
GROUP MEMBERS	:	ID	NAME
		2111540642	Nuhash Bin Hassan
		2232755642	Abu Md.Ashraful Rahat
		2311086642	Md.Shakib Rafid Shakib
		2321534042	Muhammad Farhan Rafid

Project Update Report

AI-Powered E-Commerce Platform with Intelligent Business Insights

1. Introduction

This report summarizes the development progress of the third project update for our AI-Powered E-Commerce Platform with Intelligent Business Insights. The objective of this phase was to move beyond the initial frontend-backend-chatbot connection and strengthen the practical e-commerce functions, backend API structure, access control, and product-aware AI assistance.

The team continued development through separate GitHub branches. Shakib expanded the React user interface, Rahat developed the Django data models and REST API layer, and Farhan improved the customer chatbot so that it can retrieve verified product information directly from the project database. The completed work now provides a clearer path toward integrating the independently developed modules into one stable application.

2. Frontend Development Progress

The frontend was developed using React with Vite. Shakib created and organized pages for the home screen, registration, login, user profile, product listing, product details, category listing, category details, shopping cart, checkout, order history, order details, AI chat, and invalid routes. React Router was used to connect these pages through structured client-side routes, and a reusable navigation bar was maintained across the application.

Basic shopping flow behavior was implemented through React state. Users can add products to the cart, increase quantities, remove items, proceed to checkout, place an order, and view order-related pages. The chatbot interface sends a JSON request to the Django endpoint at `/api/ai/chat/` and renders the reply returned by the backend. The corrected frontend was tested locally through the Vite development server and successfully displayed database-aware chatbot responses.

3. Backend and REST API Development Progress

Rahat organized the Django backend into modular applications for accounts, store, cart, orders, chat, and core services. Database models cover user profiles, categories, products, product images, inventory records, carts and cart items, orders, payments, invoices, chat sessions, chat messages, human-support sessions, application settings, and notifications. Serializers were added to convert these records into API-ready JSON representations.

Django REST Framework ViewSets and router-based URLs were prepared for profiles, categories, products, inventory, carts, cart items, orders, payments, invoices, chat sessions, messages, human-support requests, settings, and notifications. JWT login and refresh endpoints were configured, together with session authentication and CORS access for the Vite frontend. Permission classes restrict private records to their owners while allowing staff or support agents to manage authorized resources. Docker configuration and backend documentation are also present for repeatable development setup.

4. AI Chatbot and Product Database Integration

Farhan maintained a separate Django chatbot application connected to the local Ollama service using the phi3:mini model. The chatbot endpoint accepts POST requests at `/api/ai/chat/` and returns generated replies to the React interface. This local-model approach avoids dependence on a paid cloud language-model service during development.

For this update, `chatbot/views.py` was extended to query the existing Django Product model before sending a prompt to Ollama. The chatbot now retrieves only active products that are currently in stock. It can list all available products, show products below or above a requested price, search within a price range, and return the price, description, and current stock of a specifically named product. Questions unrelated to products continue to the Ollama model, preserving the general conversational function.

An Electronics category and five sample products were added through Django Admin: Wireless Mouse, Laptop Stand, USB-C Charger, Gaming Headset, and Mechanical Keyboard. Tests confirmed correct replies for listing products, filtering products under Tk 2000, and requesting details about the Wireless Mouse. The feature worked through both direct PowerShell API testing and the React chatbot page. Django system checking reported no issues, and the change was committed to the FarhanRafid branch as `d42c0ae`.

5. Current Project Status and Testing

The project currently contains a functional React storefront prototype, a modular Django backend with database models and REST API definitions, and a working customer AI chatbot connected to real product data. The demonstrated communication flow is React frontend to Django chatbot API to Product database, with Ollama used as a fallback for non-product questions. This verifies that the customer assistant can provide factual store information instead of inventing product names or prices.

The three development branches have not yet been fully merged into one final branch, and Rahat's complete API set still requires systematic browser and Postman verification after integration. The sample products currently exist in a local SQLite database and therefore do not transfer automatically through Git. These limitations are recorded to avoid presenting unfinished integration work as complete.

6. Future Work

The next development phase will focus on merging the latest frontend, backend, and chatbot changes while resolving any conflicts and pending migrations. The product, cart, checkout, account, and order pages will be connected to Rahat's REST APIs instead of relying only on local React state. A reusable product fixture or seed command will be added so every member can load identical demonstration data without uploading `db.sqlite3`.

Further work will include creating a formal Postman collection, testing authentication and permissions for each endpoint, improving error handling, updating the project README, and preparing the one-minute demonstration video. After the core shopping workflow is stable, the group will continue with the planned admin analytics chatbot and human-support takeover features.